Assignment 1: Black Box Testing

Hide Assignment Information

Group Category

Assignment Groups

Group Name

CS4472A - Group - 8

Instructions

CS 4472B Software Specification, Testing, and Quality Assurance

GenAI was used to augment learning outcomes.

Setup your CI within the first 3-4 days.
All members should install the GitHub app and a direct messaging app on their phones to keep up with their teammates and with pull-request approvals, you should aim to accept progress pull-requests within minutes of their creation, not days!!!

The timeline is intentionally compressed to show that software development is not a "laid back" activity.

Assignment 1: Black Box Testing

Course repository

The ATM system under test lives in the public course repository:

https://github.com/grimasta/WesternCS4472-ATM-Assignment

Every team works in its own copy of this repository (see *Submission via GitHub* below).

Late Penalties

There will be a **10% per day** late penalty for the first 2 days after the deadline. The penalty for the third day of the grace period will be **20%.** If you submit one day late your maximum grade will be reduced by 10%, two days late by 20%, three days late

by 40%. Given the extraordinary penalty incurred for the third day of delay, you should strongly consider submitting what you have in the first 2 days.

If for any reason beyond your control submission is impossible, please contact me as soon as possible.

Group Effort

You will be working in groups of up to 4 people. Work should be split equitably between members of the group. Everyone should have a good grasp of the techniques used to carry out this assignment, so try to avoid workload splits where some individuals perform the programming part and others the write-up. Make an effort to contribute equally (as much as possible) to all parts of the assignment.

Deadline

The deadline date is on the owl.uwo.ca site for Assignment 1. Your graded pull request must be opened by midnight on the deadline date. The late penalty scheme below applies to the PR opening time.

Introduction

In this assignment you will apply three of the foundational black box techniques we have seen in class. The techniques will be applied on the ATM system, the code of which is in the course repository linked above (the same project you cloned for the lab sessions).

The focal points of your assignment will be the *withdrawal*, the *deposit*, and the *transfer* operations. Each of these operations has its own business logic for calculating banking fees for the given operation. The business logic specification for calculating the fees for each of the given operations is provided below.

Withdrawal

- If the client is a *student* and
- If the transaction is performed during the *weekday*, then there is **no fee**.
- Otherwise, the fee is **0.1%** of the amount withdrawn.
- If the client is *not a student* and
- If the *balance* of the account is less than *$1,000*, then the fee is **0.3%** of the amount withdrawn.
- If, however, the *balance* is *$1,000* or more, but less than *$5,000*, the fee is **0.1%** of the amount withdrawn.
- If the balance is *$5,000* or more, then there is **no fee**.

Deposit

- If the client is a *student* and
- If the amount deposited is more than *$50* and
    - If the *balance* of the account is more than *$500*, then the fee is **0.5%** of the amount deposited.

    - Otherwise, the fee is **0.25%** of the amount deposited.
- If, however, the amount deposited is *$50* or less and
    - If the *balance* of the account is more than *$5,000*, then the fee is **0.5%** of the amount deposited.
    - Otherwise, there is **no fee**.
- If the client is *not a student* and
- If the amount deposited is more than *$250* and
    - If the *balance* of the account is more than *$2,500*, then the fee is **0.8%** of the amount deposited.
    - Otherwise, the fee is **0.4%** of the amount deposited.
- If, however, the amount deposited is *$250* or less and
    - If the *balance* of the account is more than *$10,000*, then there is **no fee**.
    - Otherwise, the fee is **0.1%** of the amount deposited.

Transfer

- If the client is a *student* and
- If the amount transferred is less than *$200* and
    - If the *balance* of the account the money is coming from is less than *$2,000* and
    - If the *balance* of the account the money is going to is less than *$1,000*, then the fee is **0.1%** of the amount transferred.
    - Otherwise, the fee is **0.05%** of the amount transferred.
    - If, however, the *balance* of the account the money is coming from is *$2,000* or more and
    - If the *balance* of the account the money is going to is less than *$1,000*, then the fee is **0.05%** of the amount transferred.
    - Otherwise, the fee is **0.025%** of the amount transferred.
- If the amount transferred is *$200* or more, and
    - If the *balance* of the account the money is coming from is less than *$2,000* and
    - If the *balance* of the account the money is going to is less than *$1,000*, then the fee is **0.05%** of the amount transferred.
    - Otherwise, the fee is **0.025%** of the amount transferred.
    - If, however, the *balance* of the account the money is coming from is *$2,000* or more and
    - If the *balance* of the account the money is going to is less than *$1,000*, then the fee is **0.25%** of the amount transferred.
    - Otherwise, the fee is **0.125%** of the amount transferred.
- If the client is *not a student* and
- If the amount transferred is less than *$100* and
    - If the *balance* of the account the money is coming from is less than *$4,000* and
    - If the *balance* of the account the money is going to is less than *$2,000*, then the fee is **0.2%** of the amount transferred.
    - Otherwise, the fee is **0.1%** of the amount transferred.

    - If, however, the *balance* of the account the money is coming from is *$4,000* or more and
    - If the *balance* of the account the money is going to is less than *$2,000*, then the fee is **1%** of the amount transferred.
    - Otherwise, the fee is **0.5%** of the amount transferred.
- If the amount transferred is *$100* or more, and
    - If the *balance* of the account the money is coming from is less than *$2,000* and
    - If the *balance* of the account the money is going to is less than *$1,000*, then the fee is **0.2%** of the amount transferred.
    - Otherwise, the fee is **0.1%** of the amount transferred.
    - If, however, the *balance* of the account the money is coming from is *$2,000* or more and
    - If the *balance* of the account the money is going to is less than *$1,000*, then the fee is **0.5%** of the amount transferred.
    - Otherwise, the fee is **0.25%** of the amount transferred.

ATM Session

For this part, you will only test the *withdrawal* transaction (*transaction choice 1*). Failure to input the right kind of data results in the following outcomes:

- Invalid amount: throws `InvalidAmountException`.
- A valid amount for withdrawal is one that can be dispensed in $20 and $50 bills (the console version of the ATM is the one under test).
- Invalid PIN format: throws `InvalidPinFormatException`.
- A valid PIN is 5 digits.

The Testing Targets

You will perform black box testing on the following classes:

1. **atm.session.Session**: The focus here is to check for PIN format and valid amount chosen for a *withdrawal* transaction (*transaction choice 1*) as per the business logic presented above for the ATM session. The technique to use for this test is *Robust Worst Case Boundary Value Analysis* under the single fault assumption principle. In the program provided, a withdrawal is limited to the available balance of the account and the daily limit. For this assignment you must create an account with a daily limit of $1,000 and an available balance of $5,000. The daily limit of $1,000 is your upper boundary in this case.
2. **bank.utils.FeesCalculator**: The focus here is to check for the correct calculation of fees while performing withdrawal, deposit, and transfer.
3. **For withdrawal:** use *Robust Worst Case Boundary Value Analysis* to generate your test cases.
4. **For deposit:** use *Weak Robust Equivalence Class Analysis* to generate your test cases.
5. **For transfer:** use *Decision Table Analysis* to generate your test cases.

What to Run

For each of the testing targets (**atm.session.Session** and **bank.utils.FeesCalculator**), run the different test cases you have generated under the different analysis techniques as JUnit tests. For example, for **atm.session.Session** you will have one class containing the tests for the PIN format and the withdrawal amount. Similarly, for **bank.utils.FeesCalculator** you will have one class for each of the three transactions: one for *withdrawal*, one for *deposit*, and one for *transfer*.

Submission via GitHub

Your graded submission for this assignment is a **pull request** in your team's own repository, not an archived file. The full workflow is specified in `TESTING.md` in the course repository; read it first. The steps below are the ones specific to Assignment 1.

1. Set up your team's repository. Clone the course repository, then create your own repository for your team (private is fine; add the instructor and the TA as collaborators so it can be graded) and push the code there. Create a `dev` branch; `main` stays protected for the whole term.

2. Write the CI workflow yourselves. Your first commit to your repository is `.github/workflows/ci.yml`, which you write from scratch. It is not provided. A GitHub Actions workflow is a YAML file describing *when* something runs (triggers) and *what* runs (jobs). Each job runs on a fresh virtual machine (`runs-on: ubuntu-latest`) and is a sequence of steps; the two building blocks you need are `actions/checkout` (gives the job your code) and `actions/setup-java` (installs Java 11, Temurin distribution), after which a step can run any shell command such as `mvn -B checkstyle:check` or `mvn -B test`. Your workflow must implement the behavior specified in `TESTING.md`: a lint job that always blocks merging, and a tests job that reads the PR's labels and requires exactly one of `phase:tests` or `phase:fix`, running the suite in the mode each phase demands. The required *behavior* is the specification; the implementation is yours. Start from the GitHub Actions documentation.

3. Configure branch protection. On your repository, protect `main`: require pull requests before merging, require `team size - 1` approving reviews (every teammate except the author), and dismiss stale approvals when new commits are pushed.

4. Do the work on `dev`. Push regularly; every push runs CI in informational mode.

5. Open the graded pull request. Open a PR from `dev` into `main`, apply exactly one label (`phase:tests` for this assignment), and fill in the PR description using the repository's pull request template. CI will run Checkstyle (must pass), the test suite (failures are posted as a comment and do *not* block merging in this phase; your tests are expected to fail), and a coverage report (informational).

6. Review and merge. Every teammate except the author approves, then merge. An unmerged PR is not a submission.

7. Tell me where it is. Submit on OWL the URL of your team's repository and a link to your merged pull request. Without this, your work cannot be found or graded.

What to Deliver

Everything you are graded on lives in your merged `phase:tests` pull request:

1. **The test cases you have created**, as a Markdown document in the PR (e.g. `docs/assignment1.md`). This part of the deliverable will have two sections. In the first section you will need to comment on the process and the assumptions you have used to calculate the test cases, using the analysis technique for the corresponding case (e.g. Robust Worst Case Boundary Value Analysis for PIN format, see *The Testing Targets* above). In the second section you will have the tables with the test cases.
2. **The test results.** This part of the deliverable will provide a table indicating the test case and whether the test passed or failed (in the same document, and reflected in the CI run on the PR).
3. **Fault localization.** For every test case that failed, state which fault in the code you believe it exposes: name the class and method (and, where possible, the specific condition or statement) and give a one-sentence justification linking the observed failure to that fault. Where several failing test cases expose the same fault, group them under one entry. Use the fault-localization table in the pull request template for this.
4. **The JUnit code** with which you ran your tests, committed in the PR under `src/test/java`.

Keep a copy of your failure log (failure ID, which test case exposed it, and a rough record of time spent testing before each failure was found and time spent diagnosing it). You will need this log in Assignment 3.

Important Note

As this is a testing course and we are trying to learn how to write good tests (meaning tests that fail and thus locate a fault in the code), there will be some faults in the code. You are not required to fix the faults, as this would alter the results of your tests.

Due on Nov 6, 2026 11:59 PM

Available on Oct 19, 2026 12:01 AM. **Hidden before availability starts.**

Submit Assignment

Files to submit

(0) file(s) to submit

After uploading, you must click Submit to complete the submission.

Add a File Record Audio Record Video

Comments